

UNIT I

1

Applet

Syllabus

Applet Basics - Introduction, limitations of AWT, Applet architecture - HTML APPLET tag - Passing parameter to Appletget, DocumentBase() and getCodeBase() , Japplet : Icons and Labels Text Fields Buttons, Combo Boxes, Checkboxes, Tabbed Panes, Scroll Panes, Trees : Tables.

Contents

- 1.1 Introduction
- 1.2 Applet Architecture
- 1.3 HTML APPLET Tag
- 1.4 Creating and Executing Applet
- 1.5 Passing Parameter to Applet
- 1.6 getDocumentBase() and getCodeBase()
- 1.7 Adding Controls to Applet
- 1.8 Multiple Choice Questions

1.1 Introduction

- Applets are the small Java programs that can be used in internetworking environment.
- These programs can be transferred over the internet from one computer to another and can be displayed on various web browsers.
- Various applications of applets are in performing arithmetic operations, displaying graphics, playing sounds, creating animation and so on.
- Following are the **situations in which we need to use applet** -
 1. For displaying the dynamic web pages we need an applet. The dynamic web page is a kind of web page on which the contents are constantly changing. For example an applet that can represent the sorting process of some numbers. During the process of sorting the positions of all the elements is changing continuously.
 2. If we want some special effects such as sound, animation and much more then the applets are used.
 3. If we want a particular application should be used by any user who might be located remotely. Then in such situation the applets are embedded into the web pages and can be transferred over the internet.

Difference between Applet and Application

Sr. No.	Applets	Applications
1.	Applets do not have main method . On loading of applets some methods of applet class get called automatically.	Application programs have main method . Within the main method the call to another methods of Java class is given.
2.	Applets cannot run independently . They can be either embedded in web page or can be run using appletviewer.	Applications program run independently .
3.	Applets cannot be read from file . Similarly applets can not write to files.	Application programs make use of I/O functions and can read a file or write to file.
4.	Applets cannot communicate with others on the network.	Java programs can communicate with other programs in distributed environment.
5.	Applets cannot execute any program on local computer.	Applications can execute a program on local computer.

Review Question

1. Differentiate applet and application with any four points.

1.2 Applet Architecture

- There are various methods which are typically used in applet for initialization and termination purpose. These methods are :
 1. Initialization
 2. Running state
 3. Idle state
 4. Dead or destroyed state (Refer Fig. 1.2.1 for applet's life cycle)

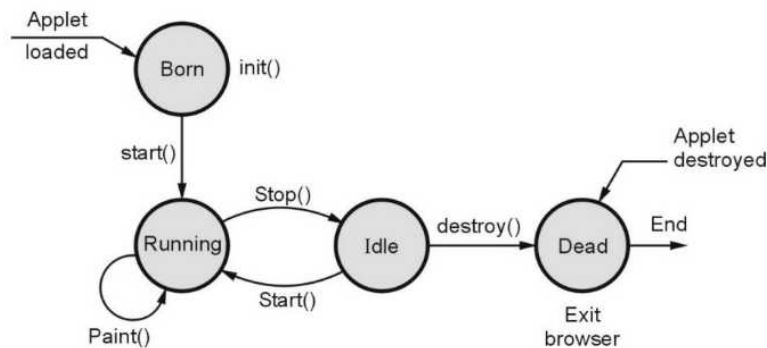


Fig. 1.2.1 Applet's life cycle

- When applet begins, the AWT calls following methods in sequence -
 - a) `init()`
 - b) `start()`
 - c) `paint()`
- When applet is terminated following method are invoked in sequence.
 - a) `stop()`
 - b) `destroy()`

1. Initialization state

- When applet gets loaded it enters in the initialization state. For this purpose the **`init()`** method is used. In this method you can initialize the required variables. This method is called only once initially at the execution of the program. The syntax can be,

```
public void init()
{
    //initialization of variables
}
```

- In this method various tasks of initialization can be performed such as -
 1. Creation of objects needed by applet
 2. Setting up of initial values
 3. Loading of image
 4. Setting up of colors.

2. Running state

- When the applet enters in the running state, it invokes the **start()** method of Applet class.
- This method is called only after the init method. After stopping the applet when we restart the applet at that time also this method is invoked. The syntax can be

```
public void start()  
{  
...  
}
```

3. Display state

- Applet enters in the display state when it wants to display some output. This may happen when applet enters in the running state. The **paint()** method is for displaying or drawing the contents on the screen. The syntax is

```
public void paint(Graphics g)  
{  
...  
}
```

- An instance of Graphics class has to be passed to this function as an argument. In this method various operations such as display of text, circle, line are invoked.

4. Idle state

- This is an idle state in which applet becomes idle. The **stop()** method is invoked when we want to stop the applet. When an applet is running if we go to another page then this method is invoked. The syntax is

```
public void stop()  
{  
...  
}
```

5. Dead state

- When applet is said to be dead then it is removed from memory. The method **destroy()** is invoked when we want to terminate applet completely and want to remove it from the memory.

```
public void destroy()
{
    ...
}
```

- It is a good practice to call stop prior to destroy method.

Review Questions

- Describe following states of applet life cycle :
 - Initialization state.
 - Running state.
 - Display state.
- Explain applet life cycle with suitable diagram.

1.3 HTML APPLET Tag

- The <APPLET> tag can be specified with the help of various attributes. These attributes help to integrate the applet into overall design of the web page.
- Various attributes of APPLET tag are -

Attribute	Description
CODE = appletfilename	The specified applet can be loaded in the web page. This attribute must be specified in order to embed the applet in the HTML file.
WIDTH = pixels HEIGHT = pixels	This attribute specifies the width and height of the applet.
ALIGN = alignment	This attribute is for specifying the alignment. Various alignments are - TOP, BOTTOM, LEFT, RIGHT, MIDDLE, ABSMIDDLE, ABSBOTTOM, TEXTTOP and BASELINE. This is an optional attribute.
CODEBASE = codebase_URL	It specifies the name of the directory in which the applet is stored. This attribute is required when the attribute is not there in the current working directory. This is an optional attribute.

HSPACE = pixels	When ALIGN is either LEFT or RIGHT this attribute is used. It specifies the amount of horizontal blank spaces the browser should leave around the applet. This is an optional attribute.
VSPACE = pixels	When ALIGN is either TOP or BOTTOM this attribute is used. It specifies the amount of vertical blank spaces the browser should leave around the applet. This is an optional attribute.
ALT = alternate text	The non Java browser can display the alternate text in place of applet. This is an optional attribute.

Review Questions

1. Explain all attributes available in `<applet>` tag.
2. Explain `<applet>` tag with its major attributes only.
3. Describe the following attributes of applet :
(i) Codebase (ii) Alt (iii) Width (iv) Code
4. Explain any four applet tag.

1.4 Creating and Executing Applet

1.4.1 Creating an Applet

- Normally the applet code makes use of two classes - **Applet** and **Graphics**.
- For the **Applet** class the package **java.applet** is required. This class provides the applet's life cycle method such as **init()**, **start()** and **paint()**.
- The **Graphics** class is supported by the package **java.awt**.
- Here is a simple applet.

Example Program

```

/*
This is my First Applet program
*/
import java.awt.*;
import java.applet.*;
public class FirstApplet extends Applet
{
    public void paint(Graphics g)
    {
        g.drawString("This is my First Applet",50,30);
    }
}

```

Program explanation :

- In above given small applet program, the main intension is to display the message “This is my First Applet”. Let us start from the beginning of the program -
- The first three lines represent a comment statement.
- Then next comes

```
import java.awt.*;  
import java.applet.*;
```

- We always need these two packages to be imported in the program. The **java.awt** package consists of **java awt** classes. Here **awt** stands for **abstract window toolkit**. The **AWT** provides the support for window based graphical interface such as for drawing screen, windows, buttons, text boxes, menus and so on. The other imported package is **java.applet**. This is essential because we need to use **Applet** class in our applet program which is included in **java.applet** package.
- The functionalities that are required to run applet inside the web browser are supported by **java.applet**.
- Then comes

```
public class FirstApplet extends Applet
```

- The class *FirstApplet* is a subclass of class *Applet*. Hence the keyword **extends** is used. Java requires that your applet subclass (here it is *FirstApplet*) should be declared as public. Hence is the declaration !
- We have then defined a method

```
public void paint(Graphics g)
```

- This method is used to paint something on the screen and it can be text, line, circle, rectangle or anything. Thus **paint** is a method which provides actual appearance on the screen. And this method requires a parameter to be passed as an object of class **graphics**. Therefore an object *g* of class **Graphics** is passed.
- Using this object the method **drawString** of *Graphics* class is invoked. [Note that *Graphics* class is a part of *java.awt* package].

```
g.drawString("This is my First Applet",50,30);
```

- To method **drawString**, firstly we have passed a string which we want to be displayed, then 50 and 30 represents the position of the string on the screen i.e. x and y positions respectively.

1.4.2 Executing an Applet

- There are two methods of executing an applet program -
 1. Adding Applet to HTML File
 2. Using Appletviewer tool
- Let us discuss these methods in detail.

1.4.2.1 Adding Applet to HTML File

- Following are the steps that need to be followed for adding applet to HTML file.

Step 1 : Write a Java applet program in a notepad. The code is as follows :

Java Program[FirstApplet.java]

```
/*
This is my First Applet program
*/
import java.awt.*;
import java.applet.*;
public class FirstApplet extends Applet
{
    public void paint(Graphics g)
    {
        g.drawString("This is my First Applet",50,30);
    }
}
```

Step 2 : Compile your applet source program using **javac** compiler, i.e.

```
D:\test>javac FirstApplet.java
```

Step 3 : Write following code in notepad/wordpad and save it with filename and extension **.html**. For example following code is saved as

Exe_FirstApplet.html, The code is

```
<html>
<body>
    <applet code="FirstApplet" width=300 height=100>

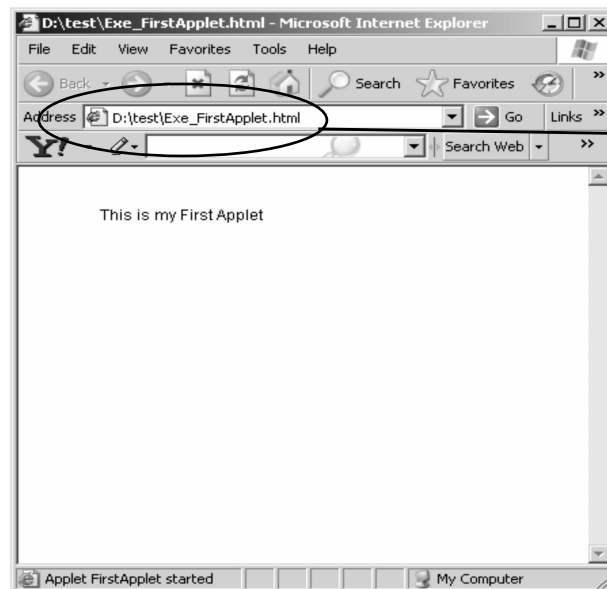
    </applet>
</body>
</html>
```

- For embedding the applet in the HTML document the **<APPLET>** tag is used. The **<APPLET>** tag supplies the name of the applet to be loaded. Suppose we want to load the applet **FirstAppletprogram** in the web page then the applet tag will be,

```
<APPLET CODE= FirstAppletprogram.class
    WIDTH = 500
    HEIGHT = 300
</APPLET>
```

- The HTML code tells the web browser to load the compiled code of applet FirstAppletprogram.class. Note that this class file must be present in the same directory where the web page is stored.

Step 4 : Load html file with some web browser, This will cause to execute your html file. It will look like this –



Note this
URL

1.4.2.2 Embedding Applet Code in Java

- We can embed the applet code in Java using comment statements. It is as shown in following illustration.

Step 1 : Write Java Program as given below.

```
/*
This is my First Applet program
*/
import java.awt.*;
import java.applet.*;
/*
<applet code="FirstApplet" width=300 height=100>
</applet>
*/
public class FirstApplet extends Applet
```

Embedded Applet in
Java

```
{
    public void paint(Graphics g)

    {
        g.drawString("This is my First Applet",50,30);
    }
}
```

- In above code we have added applet code at the beginning of the program.

```
<applet code="FirstApplet" width=300 height=100>
</applet>
```

- This will help to understand Java that the source program is an applet with the name FirstApplet.

Step 2 : By this edition you can run your applet program merely by **Appletviewer** command.

```
D:\test>javac FirstApplet.java
D:\test>Appletviewer FirstApplet.java
```

And we will get



Example 1.4.1 *Define applet. Write a program to create an applet to display message "Welcome to java applet".*

Solution : FirstApplet.java

```
/*
This is my First Applet program
*/
import java.awt.*;
import java.applet.*;
/*
<applet code="FirstApplet" width=300 height=100>
</applet>
*/
public class FirstApplet extends Applet
```

```
{  
    public void paint(Graphics g)  
    {  
        g.drawString("Welcome to Java Applet",50,30);  
    }  
}
```

Output

1.5 Passing Parameter to Applet

- Parameters are passed to applets in NAME=VALUE pairs in <PARAM> tags between the opening and closing APPLET tags.
- There can be any number of <PRAM> tags inside APPLET tag.
- Inside the applet, you read the values passed through the PARAM tags with the **getParameter()** method of the **Applet** class.
- The program below demonstrates this idea -

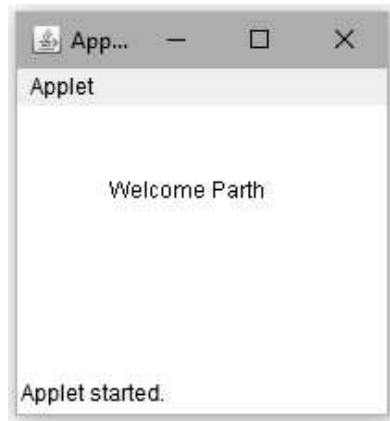
Example 1.5.1 *How can parameters be passed to an applet ? Write an applet to accept user name in the form of parameter and print 'Hello <username >'.*

Solution :

```
import java.applet.Applet;  
import java.awt.Graphics;  
/*  
<applet code="WelcomeParam" width=200 height=200>  
<param name="Username" value="Parth">  
</applet>  
*/  
public class WelcomeParam extends Applet  
{  
    String msg="";  
    public void init()  
    {
```

```
    msg = getParameter("Username");  
    msg = "Welcome " + msg;  
}  
public void paint(Graphics g)  
{  
    g.drawString(msg,50,50);  
}  
}
```

Output



Example 1.5.2 Write an applet program that accepts two input, strings using `<Param>` tag and concatenate the strings and display it in status window.

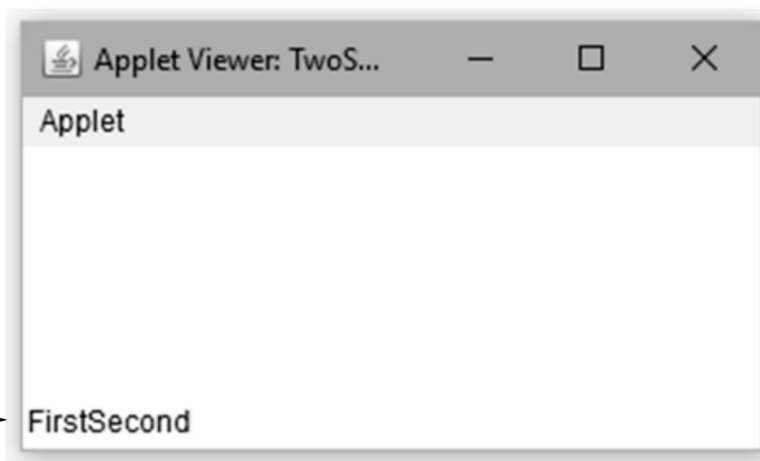
Solution : TwoStrings.java

```
import java.applet.*;  
import java.awt.*;  
  
/*  
<APPLET code="TwoStrings" width="300" height="100">  
<PARAM name="str1" value="First">  
<PARAM name="str2" value="Second">  
</APPLET>  
*/  
public class TwoStrings extends Applet  
{  
  
    public void paint(Graphics g)  
    {  
  
        String s1 = this.getParameter("str1");  
        String s2 = this.getParameter("str2");  
        String s3;
```

```
s3=s1+s2;
showStatus(s3);
}
}
```

Output

Note that the strings are displayed here



Example 1.5.3 *Design an applet which accepts username as a parameter for html page and display number of characters from it.*

Solution : Step 1 : Create HTML page as follows -

test.html

```
<html>
<head>
<title> Parameter Demo </title>
</head>
<applet code = "LengthDemo.class" width =300 height =300>
<PARAM name="uname" value="Chitra">
</applet>
</html>
```

Step 2 : Create Java program for the applet class as follows -

LengthDemo.java

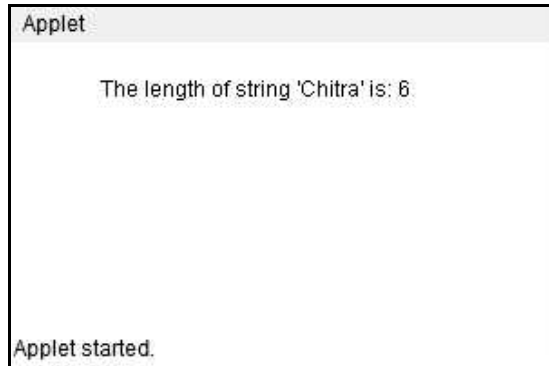
```
import java.applet.*;
import java.awt.*;
public class LengthDemo extends Applet
{
    public void paint(Graphics g)
    {
        String str = this.getParameter("uname");
        int len;
        len=str.length();
    }
}
```

```
String msg="The length of string 'Chitra' is: "+len;
g.drawString(msg,50,30);
}
}
```

Step 3 : Compile the program created in step 2 using following command on command prompt

```
D:\>javac LengthDemo.java
```

Step 4 : Open the web browser and type the name of the html file created in step 1. The output will be -

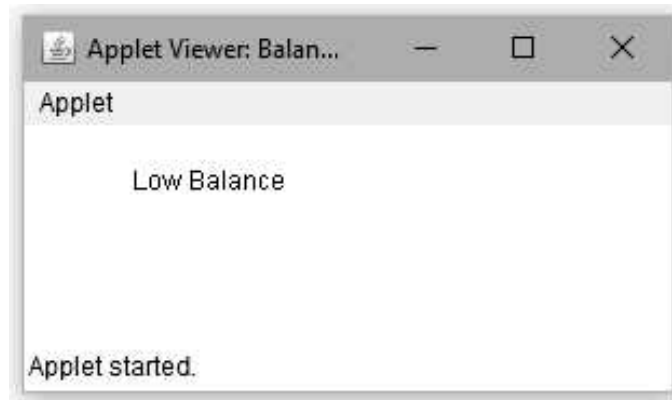


Example 1.5.4 *How to pass parameter to an applet ? Write an applet to accept Account No and balance in form of parameter and print message "low balance" if the balance is less than 500.*

Solution :

```
import java.applet.*;
import java.awt.*;

/*
<APPLET code="BalanceChk" width="300" height="100">
<PARAM name="AccNo" value=1001>
<PARAM name="Balance" value=300>
</APPLET>
*/
public class BalanceChk extends Applet
{
    public void paint(Graphics g)
    {
        int balance = Integer.parseInt(this.getParameter("Balance"));
        if(balance<500)
            g.drawString("Low Balance",50,30);
        else
            g.drawString("Sufficient Balance",50,30);
    }
}
```

Output**Review Questions**

1. Explain <PARAM> Tag of applet with suitable example.
2. Explain following methods for applet with an example :
 - 1) Passing parameter to applet
 - 2) Embedding <applet> tags in java code.

1.6 getDocumentBase() and getCodeBase()

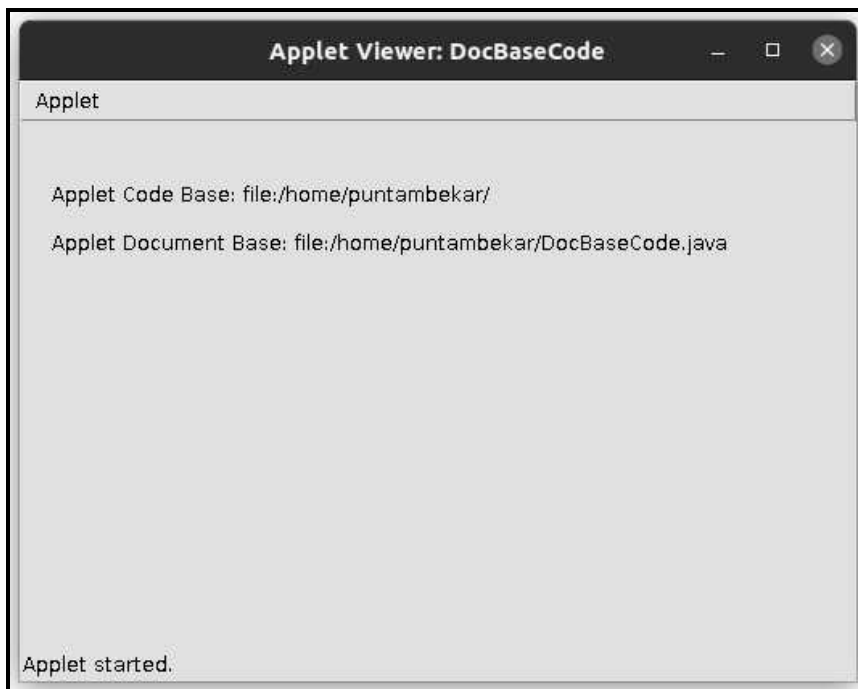
- The **getDocumentBase()** and **getCodeBase()** are the two methods that return the names of the directories. The **getDocumentBase()** is the name of the directory which contains the HTML file that starts the applet. The **getCodeBase()** is the name of the directory that contains the class file of the applet is loaded.
 - **URL getCodeBase()** : Gets the base URL.
 - **URL getDocumentBase()** : Gets the URL of the document in which the apple is embedded.
- Following applet program shows the demonstration of these two directories.

Java Program

```
import java.awt.*;
import java.applet.*;
import java.net.*;
/*
<applet code="DocBaseCode" width=500 height=300>
</applet>
*/
public class DocBaseCode extends Applet
{
    public void paint(Graphics g)
    {
```

```
String msg;  
//getCodeBase Method  
URL appletCodeDir = getCodeBase();  
msg = "Applet Code Base: "+appletCodeDir.toString();  
g.drawString(msg, 20, 50);  
  
//getDocumentBase Method  
URL appletDocDir = getDocumentBase();  
msg = "Applet Document Base: "+appletDocDir.toString();  
g.drawString(msg, 20, 80);  
}  
}
```

Output



1.7 Adding Controls to Applet

- We can add various controls such as textbox, Push Button, Radio button, Check box to the applet. Let us understand this with illustrative examples :

1.7.1 Buttons

- The **Button** class is used to add the labeled button to the applet. The caption of the button is passed as a parameter to it. The syntax is

```
Button button_object=new Button(caption string)
```

- Using **add** method we can add the button control on the applet window. Following program shows how to add button to the applet.

Example Program

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="Button1" width=350 height=200>
</applet>
*/
public class Button1 extends Applet
{
    Button button=new Button("Click Me");
    public void init()
    {
        add(button);
    }
}
```

Output



1.7.2 Text Fields

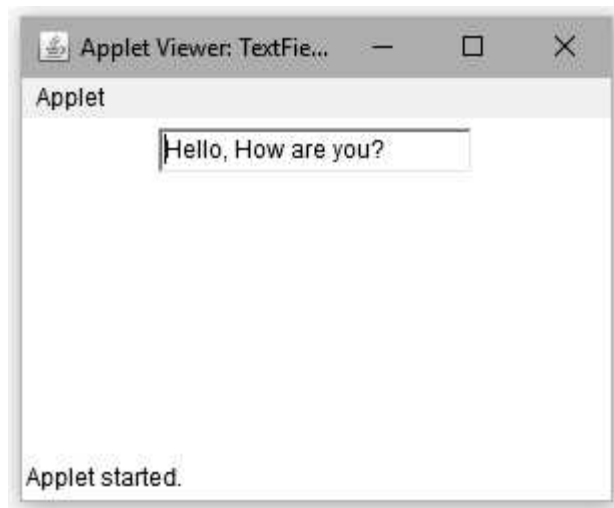
- The **TextField** class allows to add the single line text. Following program shows how to add a textfield control to the applet.

Example Program

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="TextFieldDemo" width=350 height=200>
</applet>
*/
```

```
public class TextFieldDemo extends Applet
{
    TextField tf=new TextField("Hello, How are you?");
    public void init()
    {
        add(tf);
    }
}
```

Output



1.7.3 Combo Boxes

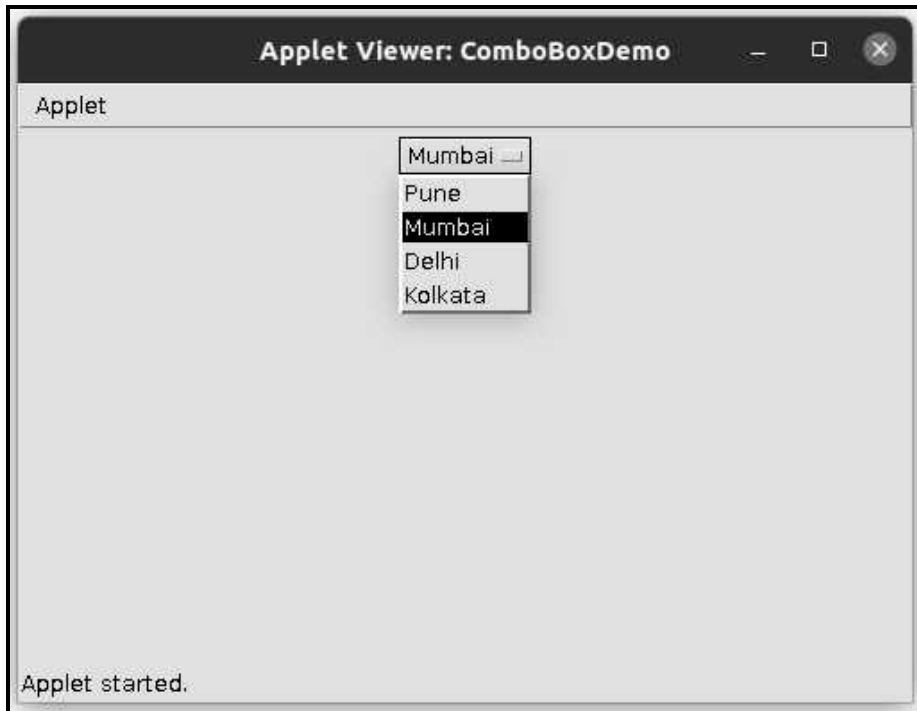
- Combobox is a dropdown list. It forces the user to select only one element from the list. It is easier to change the element of the list. It is basically a Choice control. Hence it is created using **Choice()** class.
- Following program demonstrates the use of combox control.

Java Program[ComboBoxDemo.java]

```
import java.applet.*;
import java.awt.*;
/*
<applet code="ComboBoxDemo" width=350 height=200>
</applet>
*/
public class ComboBoxDemo extends Applet
{
    Choice city=new Choice();
    public void init()
    {
```

```
city.add("Pune");  
city.add("Mumbai");  
city.add("Delhi");  
city.add("Kolkata");  
add(city);  
}  
}
```

Output



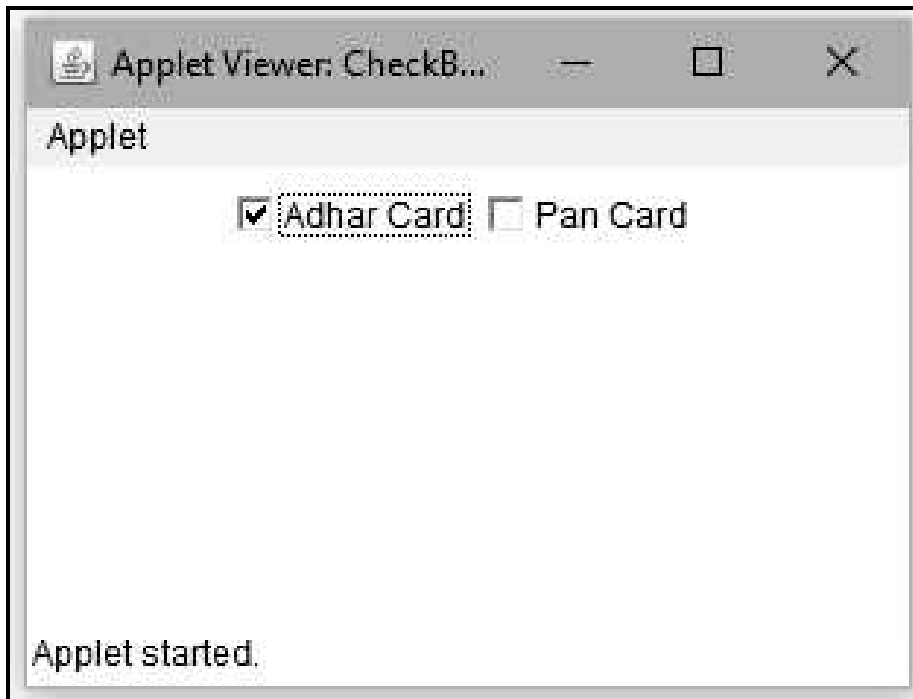
1.7.4 Checkboxes

- The Checkbox class is used to create check boxes. It is used to turn an option true or false. Following program shows the use of checkbox.

Example Program

```
import java.awt.*;  
import java.awt.event.*;  
import java.applet.*;  
/*  
<applet code="CheckBoxDemo" width=350 height=200>  
</applet>  
*/  
public class CheckBoxDemo extends Applet  
{
```

```
Checkbox cb1=new Checkbox("Adhar Card",true);
Checkbox cb2=new Checkbox("Pan Card");
public void init()
{
    add(cb1);
    add(cb2);
}
}
```

Output**1.8 Multiple Choice Questions**

Q.1 On which side applet always executed ?

- ☐ a Server side
- ☐ b Client side

Q.2 Which method of the Applet class displays the result of applet code on screen ?

- | | |
|--|---|
| ☐ a run() method | ☐ b paint() method |
| ☐ c drawString() method | ☐ d main() method |

Q.3 Applet can be embedded in _____.

- | | |
|--|--|
| ☐ a HTML document | ☐ b word document |
| ☐ c gif file | ☐ d rtf file |

Q.4 Which of the following is true about applet ?

- ☐ a Applets do not have main() method.
- ☐ b Applets must run under appletviewer or web browser.
- ☐ c The user I/O is not performed using Java's stream I/O class.
- ☐ d All of these.

Q.5 Executable applet is _____.

- ☐ a .applet file
- ☐ b .java.html
- ☐ c .java file
- ☐ d .class file

Q.6 Which object can be constructed to show any number of choices in the visible window ?

- ☐ a Labels
- ☐ b Choice
- ☐ c List
- ☐ d Checkbox

Q.7 The drawString method is defined in _____.

- ☐ a java.awt
- ☐ b java.applet
- ☐ c java.io
- ☐ d java.util

Q.8 Which class provides many methods for graphics programming ?

- ☐ a java.awt
- ☐ b java.Graphics
- ☐ c java.awt.Graphics
- ☐ d None of these

Q.9 When we invoke repaint() for a java.awt.Component object, the AWT invokes the method :

- ☐ a draw()
- ☐ b update()
- ☐ c show()
- ☐ d paint()

Q.10 Which method executes only once ?

- ☐ a start()
- ☐ b stop()
- ☐ c init()
- ☐ d destroy()

Q.11 What does the following line of code do ?

Textfield text = new Textfield(20);

- ☐ a Creates text object that can hold 20 rows of text.
- ☐ b Creates text object that can hold 20 columns of text.
- ☐ c Creates the object text and initializes it with the value 20.
- ☐ d This is invalid code

Answer Keys for Multiple Choice Questions :

Q.1	b	Q.2	b	Q.3	a
Q.4	d	Q.5	d	Q.6	c
Q.7	a	Q.8	c	Q.9	b
Q.10	c	Q.11	b		

